Sai Judala

# Module 11: Test Plan

## 1. Introduction

This document covers the testing strategy for the Order Management API, a RESTful API built with Node.js and Express.js. The API handles customer orders and supports creating, reading, updating, and deleting orders through 5 endpoints. Data is stored using SQLite, which keeps records between server restarts. The goal of this test plan is to verify that each endpoint works correctly, handles bad input properly, and returns the right HTTP status codes and response data.

## 2. Testing Scope

The following endpoints are covered in this test plan:

- POST /orders - Create a new order
- GET /orders - Retrieve all orders
- GET /orders/:id - Retrieve a specific order by ID
- PATCH /orders/:id - Update the status of an order
- DELETE /orders/:id - Delete an order

Testing also covers error handling for missing fields, invalid status values, and requests made with IDs that do not exist in the database.

## 3. Testing Approach

Three types of testing are used in this plan:

### Unit Testing

Unit tests check that individual endpoint functions handle input correctly on their own, without relying on external dependencies like a live database. The focus is on input validation, making sure the API returns the right error responses when required fields are missing or when an invalid value is passed in.

### Integration Testing

Integration tests verify that the API endpoints work correctly when connected to the database. These tests cover the full CRUD lifecycle, creating an order, retrieving it, updating its status, deleting it, and then confirming it is no longer accessible. An in-memory SQLite database is used so tests run cleanly without touching the real data file.

### Postman Testing

Postman tests validate the API from an end-user perspective by sending real HTTP requests and checking the responses. Each request has test scripts that verify the status code, response

body structure, required fields, and response time. These tests were run against the locally hosted server on http://localhost:3000.

## 4. Test Environment

- Operating System: macOS
- Runtime: Node.js v20
- Framework: Express.js
- Database: SQLite (file-based via better-sqlite3, in-memory for automated tests)
- Testing Tools: Jest v29, Supertest v6, Postman
- Server URL: http://localhost:3000

## 5. Test Deliverables

- This test plan document
- Jest unit and integration test file (app.test.js)
- Postman collection with test scripts added to each request
- Screenshots of Postman test results
- Jest terminal output showing all tests passing

## 6. Unit Test Cases

The following table documents the unit test cases designed to verify input validation and error handling at the endpoint level.

| Test Case ID | Description | Input | Expected Output | Pass/Fail Criteria |
|---|---|---|---|---|
| UT-001 | POST /orders - missing customerName | { item: "Laptop", quantity: 1 } | 400 Bad Request with error message | Passes if status is 400 and error message is returned |
| UT-002 | POST /orders - missing item | { customerName: "Jane", quantity: 1 } | 400 Bad Request with error message | Passes if status is 400 and error message is returned |
| UT-003 | POST /orders - missing quantity | { customerName: "Jane", item: "Laptop" } | 400 Bad Request with error message | Passes if status is 400 and error message is returned |
| UT-004 | PATCH /orders/:id - invalid status value | { status: "InvalidStatus" } | 400 Bad Request | Passes if status is 400 |
| UT-005 | GET /orders/:id - non-existent ID | /orders/non-existent-id | 404 Not Found with error message | Passes if status is 404 and error message is returned |

| | | | | |
|---|---|---|---|---|
| UT-006 | DELETE /orders/:id - non-existent ID | /orders/non-existent-id | 404 Not Found | Passes if status is 404 |

## 7. Integration Test Cases

The following table documents the integration test cases designed to verify that the API endpoints interact correctly with the SQLite database across the full order lifecycle.

| Test Case ID | Description | Input | Expected Output | Pass/Fail Criteria |
|---|---|---|---|---|
| IT-001 | Create order with valid data | { customerName: "Jane Doe", item: "Laptop", quantity: 1 } | 201 Created with order object and transactionId | Passes if order is saved to DB and response contains id and transactionId |
| IT-002 | Retrieve all orders | GET /orders | 200 OK with array of orders | Passes if response is an array with at least 1 order |
| IT-003 | Retrieve a specific order by ID | GET /orders/:id | 200 OK with correct order object | Passes if the correct order is returned with matching ID |
| IT-004 | Update order status | { status: "Shipped" } | 200 OK with updated order | Passes if status field is updated correctly in the database |
| IT-005 | Delete an order | DELETE /orders/:id | 200 OK with success message | Passes if order is removed from the database |
| IT-006 | Retrieve deleted order | GET /orders/:id after delete | 404 Not Found | Passes if deleted order is no longer accessible |

## 8. Postman Test Cases

The following table documents all Postman test scripts added to each request in the collection. Tests were run against the local server and validate status codes, response body structure, headers, and response times.

| Request | Test | Validation | Expected Result |
|---|---|---|---|
| POST /orders | Status code check | pm.response.to.have.status(201) | 201 Created |
| POST /orders | Order object exists | jsonData.order.id exists | ID is present in response |
| POST /orders | TransactionId exists | jsonData.transactionId exists | TransactionId present from payment gateway |

Sai Judala

| | | | |
|---|---|---|---|
| POST /orders | Content-Type is JSON | pm.response.to.have.header(Content-Type) | Header is present |
| POST /orders | Response time check | responseTime below 5000ms | Under 5 seconds |
| GET /orders | Status code check | pm.response.to.have.status(200) | 200 OK |
| GET /orders | Response is array | jsonData is an array | Array returned |
| GET /orders | Content-Type is JSON | pm.response.to.have.header(Content-Type) | Header is present |
| GET /orders | Response time check | responseTime below 3000ms | Under 3 seconds |
| GET /orders/:id | Status code check | pm.response.to.have.status(200) | 200 OK |
| GET /orders/:id | Required fields present | id, customerName, status, createdAt all exist | All fields present |
| GET /orders/:id | Response time check | responseTime below 3000ms | Under 3 seconds |
| PATCH /orders/:id | Status code check | pm.response.to.have.status(200) | 200 OK |
| PATCH /orders/:id | Status updated correctly | jsonData.order.status equals "Shipped" | Status is Shipped |
| PATCH /orders/:id | Response time check | responseTime below 3000ms | Under 3 seconds |
| DELETE /orders/:id | Status code check | pm.response.to.have.status(200) | 200 OK |
| DELETE /orders/:id | Deletion confirmed | jsonData.message equals "Order deleted successfully." | Message confirmed |
| DELETE /orders/:id | Response time check | responseTime below 3000ms | Under 3 seconds |
| GET /orders/fake-id-999 | Status code check | pm.response.to.have.status(404) | 404 Not Found |
| GET /orders/fake-id-999 | Error message returned | jsonData.error equals "Order not found." | Error message present |
| GET /orders/fake-id-999 | Response time check | responseTime below 3000ms | Under 3 seconds |

## 9. Postman Test Results

All Postman tests were executed against the running local server. The table below summarizes the results. Screenshots of each individual test run showing the green PASSED indicators are included after the table.

| Request | Tests Run | Passed | Failed | Status Code |
|---|---|---|---|---|
| POST /orders (Create Order) | Status code, order object, transactionId, Content-Type, response time | 5 | 0 | 201 Created |
| GET /orders (Get All Orders) | Status code, array check, Content-Type, response time | 4 | 0 | 200 OK |
| GET /orders/:id (Get Order By ID) | Status code, fields check, response time | 3 | 0 | 200 OK |
| PATCH /orders/:id (Update Status) | Status code, status value, response time | 3 | 0 | 200 OK |
| DELETE /orders/:id (Delete Order) | Status code, deletion message, response time | 3 | 0 | 200 OK |
| GET /orders/fake-id-999 (Error Case) | Status code, error message, response time | 3 | 0 | 404 Not Found |
| **TOTAL** | | **21** | **0** | **All Passed** |

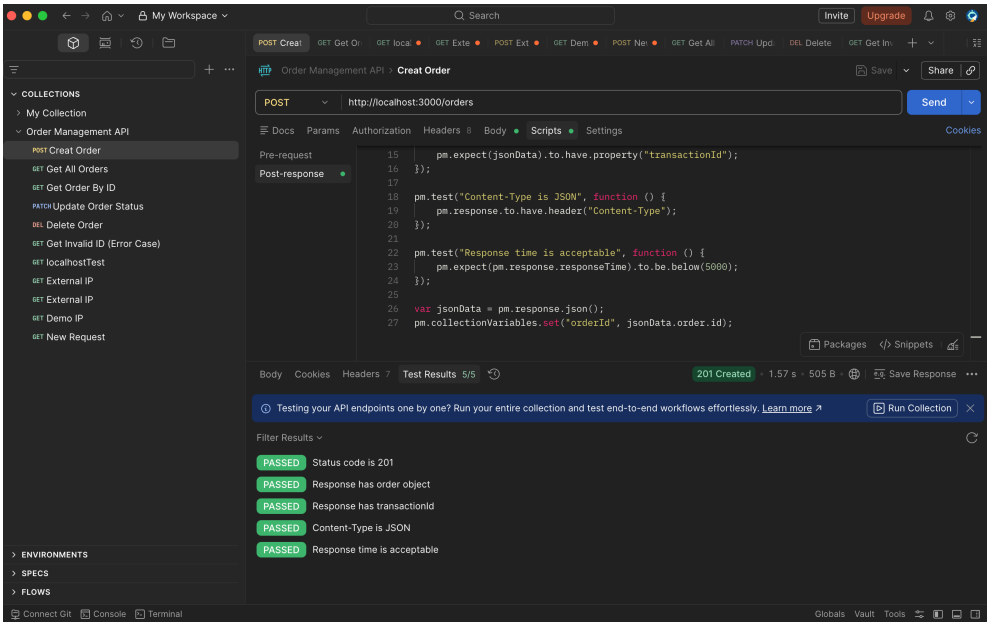


*Figure 1: POST Create Order*

Sai Judala

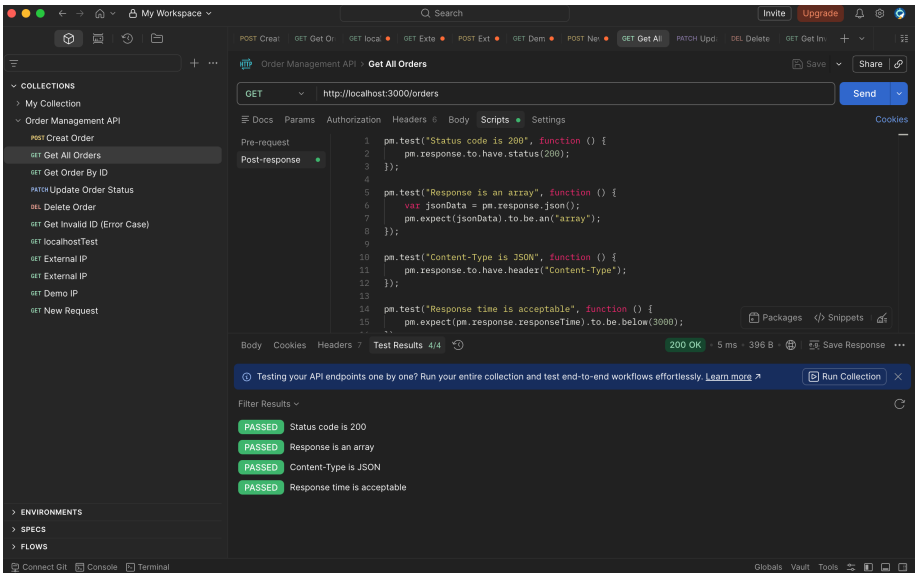


*Figure 2: GET All Orders*

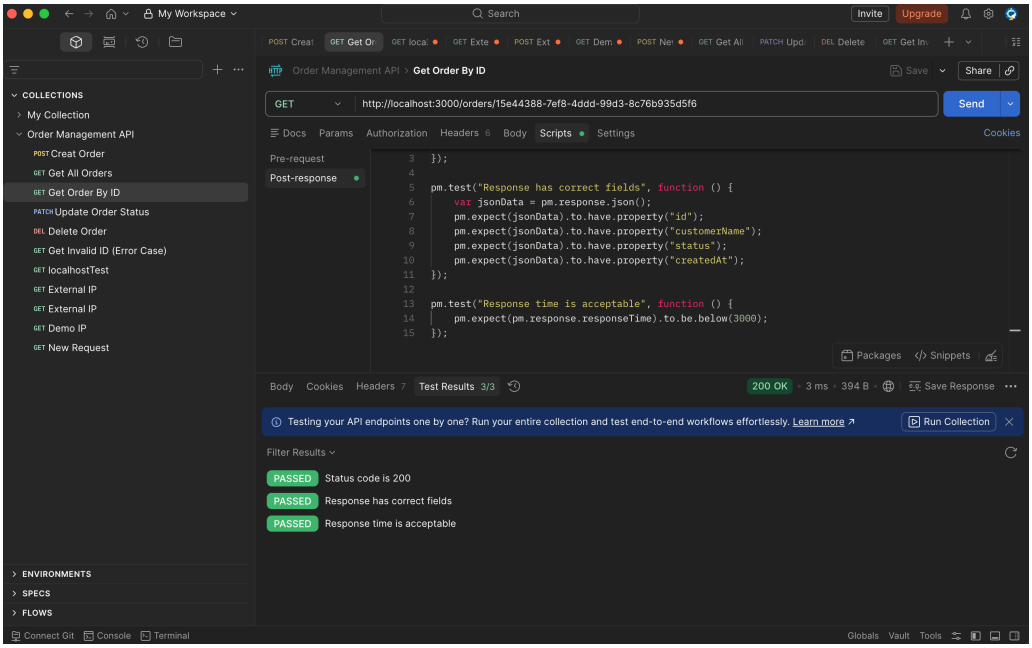


*Figure 3: GET Orders by ID*

Sai Judala

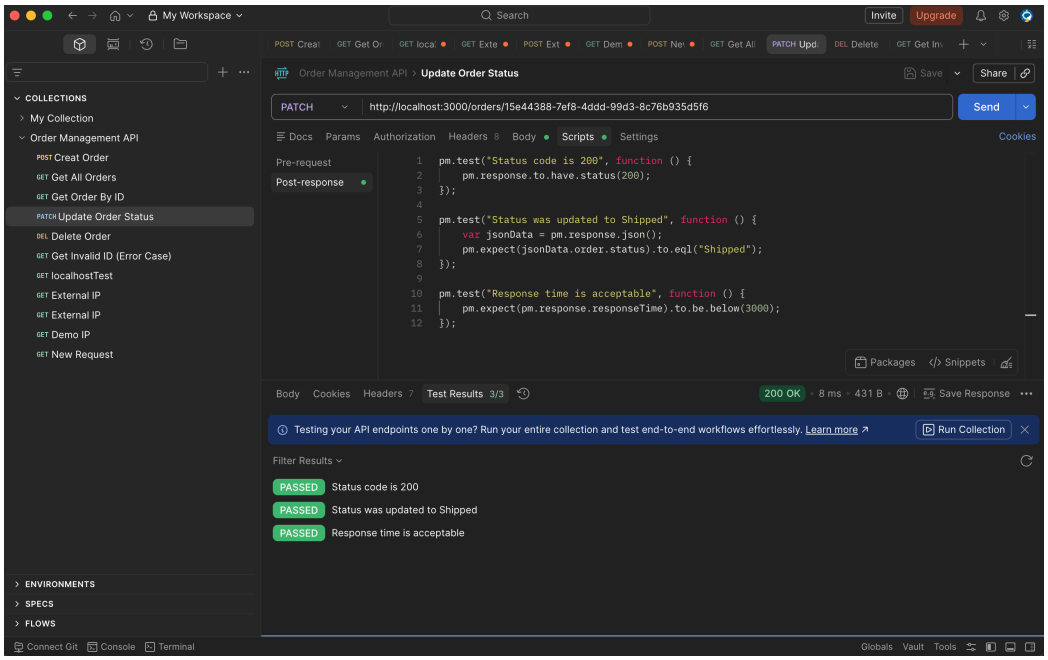


*Figure 4: PATCH Update Order Status*

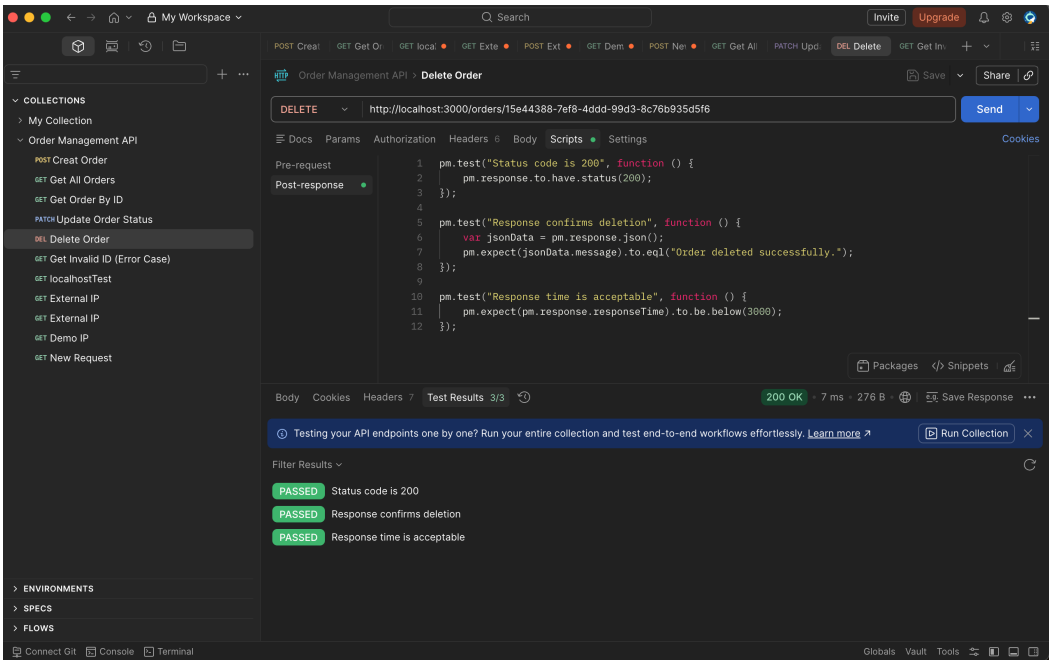


*Figure 5: DELETE Order*

Sai Judala

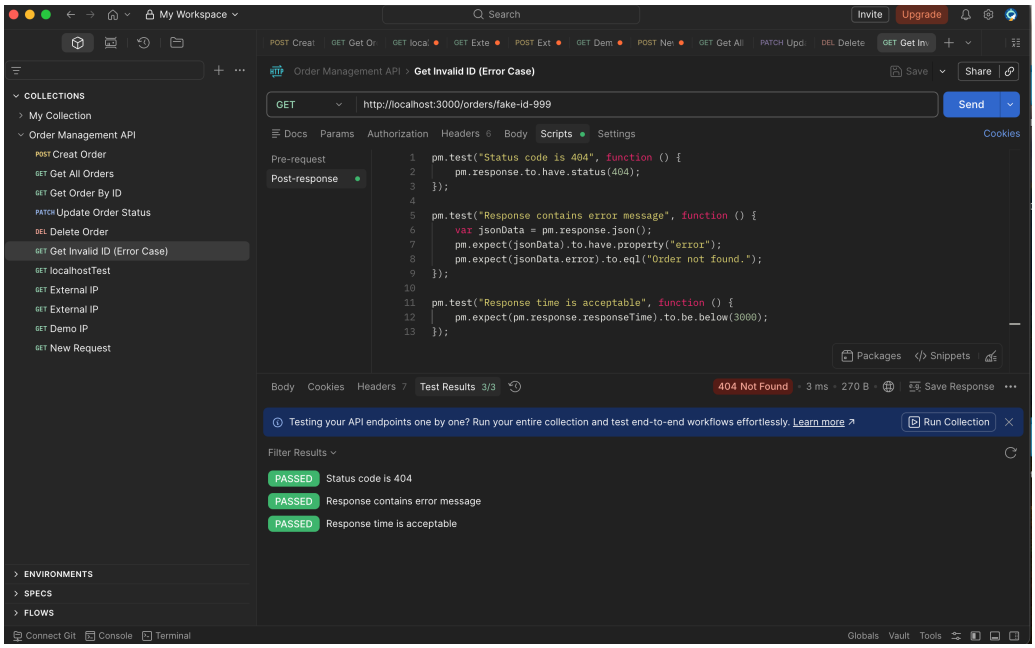

*Figure 6: GET Invalid ID*

## 10. Jest Test Results

All automated unit and integration tests were run using Jest and Supertest. The table below documents the results of each test case. All 12 tests passed with a total run time of under 1 second.

| Type | Test Name | Result | Duration |
|---|---|---|---|
| Unit | POST /orders - missing customerName returns 400 | PASSED | 15ms |
| Unit | POST /orders - missing item returns 400 | PASSED | 4ms |
| Unit | POST /orders - missing quantity returns 400 | PASSED | 1ms |
| Unit | PATCH /orders/:id - invalid status returns 400 | PASSED | 1ms |
| Unit | GET /orders/:id - non-existent ID returns 404 | PASSED | 1ms |
| Unit | DELETE /orders/:id - non-existent ID returns 404 | PASSED | 1ms |
| Integration | POST /orders - creates order successfully | PASSED | 103ms |
| Integration | GET /orders - returns array of orders | PASSED | 1ms |
| Integration | GET /orders/:id - retrieves specific order | PASSED | 1ms |
| Integration | PATCH /orders/:id - updates order status | PASSED | 1ms |
| Integration | DELETE /orders/:id - deletes order | PASSED | 1ms |
| Integration | GET /orders/:id - deleted order returns 404 | PASSED | 1ms |

| | | | |
|---|---|---|---|
| **TOTAL** | **12 Tests** | **12 Passed** | **0 Failed** |

*Figure 7: Terminal Output*

## 11. Test Summary

All tests across all three testing phases passed successfully. No failures or unexpected behaviors were encountered during testing.

- Unit Tests: 6 out of 6 passed
- Integration Tests: 6 out of 6 passed
- Postman Tests: 21 out of 21 passed

The Order Management API correctly handles valid requests, returns appropriate error responses for invalid or missing input, and interacts with the database as expected across all CRUD operations.